Assignment No. 4
EECS 348
Software Engineering I
Antonio DePaolis
Due: 11:59 PM, Thursday, October 15, 2026

1. Analyze the two GenAI programs. The analysis must include:
    a. Name of two GenAI programs
        i. CoPilot
        ii. Google Gemini
    b. How you accessed them (e.g., Google query, CoPilot on my computer)
        i. Accessed CoPilot via the Microsoft CoPilot website
        ii. Accessed Google Gemini via Google Gemini website
    c. Prompt used. (You must use the same prompt for both GenAIs.)

Using C++ create code that utilizes objects not functions to take in an input file name from the user which holds a 9 by 9 grid of numbers 1-9 and _ to represent empty slots they are separated by a space. To solve the sudoku puzzle we will use a depth first search with backtracking. k will represent the slots that are unfilled out of the 81 cells. Once the k slots are identified, deduce possible candidates for each k slot and insert it. If there are no values go back until you can find something to try until there are no more possibilities which results in no solution. Make sure this is a recursive object-oriented program. The output should be a new file with the name being the inputted file name+solution.txt. The solution file should include the puzzle file name at the top, the puzzle from the input file, then the solution to the puzzle. If there is no solution it should simply print out “No solution found”. If there is a solution print it out then under it print out “Solutions found: solution number”. If it isn’t unique make sure to print out each possible solution then after all the solutions have been printed out print “Solutions found: solution number”. Make sure to cover edge cases like input has incorrect formatting or name of input file doesn’t exist.

    d. Generated code from both GenAIs.
        i. CoPilot

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

class SudokuSolver
{
private:
   vector<vector<char>> board;
   vector<vector<vector<char>>> solutions;
   vector<pair<int, int>> emptyCells;
   string inputFileName;

public:
```

```
SudokuSolver()
{
    board.resize(9, vector<char>(9));
}

bool loadPuzzle(const string& fileName)
{
    inputFileName = fileName;

    ifstream fin(fileName);

    if (!fin)
    {
        cerr << "Error: Input file does not exist." << endl;
        return false;
    }

    string token;
    int count = 0;

    while (fin >> token)
    {
        if (count >= 81)
        {
            cerr << "Error: Too many entries in puzzle." << endl;
            return false;
        }

        char value = token[0];

        if (!((value >= '1' && value <= '9') || value == '_'))
        {
            cerr << "Error: Invalid character in puzzle." << endl;
            return false;
        }

        board[count / 9][count % 9] = value;
        count++;
    }
```

```
        if (count != 81)
        {
            cerr << "Error: Puzzle must contain exactly 81 entries." <<
endl;
            return false;
        }

        findEmptyCells();
        return true;
    }

    void findEmptyCells()
    {
        emptyCells.clear();

        for (int r = 0; r < 9; r++)
        {
            for (int c = 0; c < 9; c++)
            {
                if (board[r][c] == '_')
                {
                    emptyCells.push_back({r, c});
                }
            }
        }
    }

    bool validPlacement(int row, int col, char value)
    {
        for (int i = 0; i < 9; i++)
        {
            if (board[row][i] == value)
                return false;
        }

        for (int i = 0; i < 9; i++)
        {
            if (board[i][col] == value)
```

```
            return false;
        }

        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        for (int r = startRow; r < startRow + 3; r++)
        {
            for (int c = startCol; c < startCol + 3; c++)
            {
                if (board[r][c] == value)
                    return false;
            }
        }

        return true;
    }

    vector<char> getCandidates(int row, int col)
    {
        vector<char> candidates;

        for (char num = '1'; num <= '9'; num++)
        {
            if (validPlacement(row, col, num))
            {
                candidates.push_back(num);
            }
        }

        return candidates;
    }

    void solveDFS(int cellIndex)
    {
        if (cellIndex == emptyCells.size())
        {
            solutions.push_back(board);
            return;
```

```
    }

    int row = emptyCells[cellIndex].first;
    int col = emptyCells[cellIndex].second;

    vector<char> candidates = getCandidates(row, col);

    if (candidates.empty())
    {
        return;
    }

    for (char candidate : candidates)
    {
        board[row][col] = candidate;

        solveDFS(cellIndex + 1);

        board[row][col] = '_';
    }
}

void solve()
{
    solveDFS(0);
}

void printBoard(ofstream& fout,
                const vector<vector<char>>& currentBoard)
{
    for (int r = 0; r < 9; r++)
    {
        for (int c = 0; c < 9; c++)
        {
            fout << currentBoard[r][c];

            if (c != 8)
            {
                fout << " ";
```

```
            }
        }
        fout << endl;
    }
}

bool writeResults()
{
    string outputFile = inputFileName + "solution.txt";

    ofstream fout(outputFile);

    if (!fout)
    {
        cerr << "Error: Cannot create output file." << endl;
        return false;
    }

    fout << "Puzzle File: " << inputFileName << endl;
    fout << endl;

    fout << "Original Puzzle:" << endl;
    printBoard(fout, board);
    fout << endl;

    if (solutions.empty())
    {
        fout << "No solution found" << endl;
    }
    else
    {
        for (size_t i = 0; i < solutions.size(); i++)
        {
            fout << "Solution " << i + 1 << ":" << endl;

            printBoard(fout, solutions[i]);

            fout << endl;
        }
```

```
            fout << "Solutions found: "
                 << solutions.size()
                 << endl;
        }

        fout.close();

        cout << "Output written to "
             << outputFile
             << endl;

        return true;
    }
};

class SudokuApplication
{
public:
    void run()
    {
        string fileName;

        cout << "Enter puzzle filename: ";
        cin >> fileName;

        SudokuSolver solver;

        if (!solver.loadPuzzle(fileName))
        {
            return;
        }

        solver.solve();
        solver.writeResults();
    }
};

int main()
```

```
{
    SudokuApplication app;
    app.run();

    return 0;
}
```

ii. Gemini

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuBoard {
private:
   char grid[9][9];
   std::string fileName;

public:
   SudokuBoard() {
       for (int i = 0; i < 9; ++i) {
           for (int j = 0; j < 9; ++j) {
               grid[i][j] = '_';
           }
       }
   }

   bool loadFromFile(const std::string& name) {
       fileName = name;
       std::ifstream file(fileName);
       if (!file.is_open()) {
           return false;
       }

       for (int i = 0; i < 9; ++i) {
           for (int j = 0; j < 9; ++j) {
               if (!(file >> grid[i][j])) {
                   return false; // Insufficient data or incorrect
formatting
               }
               if (grid[i][j] != '_' && (grid[i][j] < '1' || grid[i][j]
> '9')) {
                   return false; // Invalid character
               }
           }
```

```
        }

        // Verify there is no extra trailing data
        char extra;
        if (file >> extra) {
            return false;
        }

        return true;
    }

    std::string getFileName() const {
        return fileName;
    }

    char getCell(int r, int c) const {
        return grid[r][c];
    }

    void setCell(int r, int c, char val) {
        grid[r][c] = val;
    }

    bool isValid(int row, int col, char num) const {
        // Check row and column constraints
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == num) return false;
            if (grid[i][col] == num) return false;
        }

        // Check 3x3 subgrid constraints
        int startRow = row - row % 3;
        int startCol = col - col % 3;
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[startRow + i][startCol + j] == num) return
false;
            }
        }
```

```
        return true;
    }

    void printToStream(std::ostream& out) const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                out << grid[i][j] << (j == 8 ? "" : " ");
            }
            out << "\n";
        }
    }
};

class SudokuSolver {
private:
    SudokuBoard board;
    std::vector<SudokuBoard> solutions;

    void solveRecursive(int row, int col) {
        if (row == 9) {
            solutions.push_back(board);
            return;
        }

        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        if (board.getCell(row, col) != '_') {
            solveRecursive(nextRow, nextCol);
        } else {
            for (char num = '1'; num <= '9'; ++num) {
                if (board.isValid(row, col, num)) {
                    board.setCell(row, col, num);
                    solveRecursive(nextRow, nextCol);
                    board.setCell(row, col, '_'); // Backtrack
                }
            }
        }
```

```
    }

public:
    explicit SudokuSolver(const SudokuBoard& b) : board(b) {}

    void solve() {
        solutions.clear();
        solveRecursive(0, 0);
    }

    const std::vector<SudokuBoard>& getSolutions() const {
        return solutions;
    }
};

class SudokuApp {
public:
    void run() {
        std::string inputFilename;
        std::cout << "Enter the Sudoku puzzle input file name: ";
        if (!(std::cin >> inputFilename)) {
            return;
        }

        SudokuBoard initialBoard;
        if (!initialBoard.loadFromFile(inputFilename)) {
            std::cerr << "Error: File does not exist or has incorrect
formatting.\n";
            return;
        }

        SudokuSolver solver(initialBoard);
        solver.solve();

        const auto& solutions = solver.getSolutions();
        std::string outputFilename = inputFilename + "_solution.txt";
        std::ofstream outFile(outputFilename);

        if (!outFile.is_open()) {
```

```
            std::cerr << "Error: Could not create output file.\n";
            return;
        }

        // Write puzzle file name at the top
        outFile << "Puzzle File: " << initialBoard.getFileName() <<
"\n\n";

        // Write initial puzzle
        outFile << "Initial Puzzle:\n";
        initialBoard.printToStream(outFile);
        outFile << "\n";

        if (solutions.empty()) {
            outFile << "No solution found\n";
            std::cout << "No solution found. Output written to " <<
outputFilename << "\n";
        } else {
            int solNum = 1;
            for (const auto& sol : solutions) {
                outFile << "Solution " << solNum << ":\n";
                sol.printToStream(outFile);
                outFile << "\n";
                solNum++;
            }
            outFile << "Solutions found: " << solutions.size() << "\n";
            std::cout << "Solutions found: " << solutions.size() << ".
Results written to " << outputFilename << "\n";
        }
    }
};

int main() {
    SudokuApp app;
    app.run();
    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.
    i. Correctness
        1. Copilot

- Test Execution
    a. Successfully executed with valid input and outputted correct solutions
- Edge Cases
    a. Catches incorrect input name
    b. Catches file with invalid contents
    c. Catches end of file indicator
- Error Handling
    a. States “Input file does not exist.” if inputted file does not exist
    b. States “Invalid character in puzzle.” if file has invalid contents
    c. When catching the end of file indicator it accidentally catches it due to the input name not existing so it states“Input file does not exist.”

        2. Gemini

- Test Execution
    a. Successfully executed with valid input and outputted correct solutions
- Edge cases
    a. Catches incorrect input name
    b. Catches file with invalid contents
    c. Catches end of file indicator
- Error Handling
    a. States “File does not exist or has incorrect formatting”
        i. If file does not exist
        ii. Or file has invalid content
    b. No text when catching end of file indicator just quites

ii. Execution time

1. Copilot

- Both run with constants that recurse based on number of empty space so the result is O(c^k) time complexity with c=9
- Execution time test
  - Puzzle1
    - 3 ms
    - 3 ms
    - 6 ms
  - Puzzle5
    - 5 ms
    - 5 ms
    - 6 ms
  - Both code ran fast enough that the difference is negligible
- CoPilot does run faster as instead of checking all 81 inputs it stores the locations of the empty slots and changes those only while comparing it to their row and column
- Though the speed is negligible due to the small size

2. Gemini

- Both run with constants that recurse based on number of empty space so the result is O(c^k) time complexity with c=9
- Execution time test
  - Puzzle1
    - 4 ms
    - 3 ms
    - 4 ms
  - Puzzle5
    - 5 ms
    - 3 ms
    - 3 ms
  - Both code ran fast enough that the difference is negligible
- Gemini runs slower as it will loop through the entire grid and check if its empty.
- Though the speed is negligible due to the small size

iii. Space complexity
    1. Both

Due to input size being a constant 9x9 grid of 81 elements the space complexity remains at O(1) which means it is just based on the size of how they store the grid itself. Though due to the 9x9 grid this isn't that relevant and even if it did result in multiple calls the size difference wouldn't be that different. Though just to state it Copilot will hold less space as Gemini holds the file name along with the grid resulting in taking up more space than Copilot that only stores the grid

iv. Maintainability

Readability

Gemini just has better maintainability in general due to it separating its tasks to various methods along with separating the file reader and solver into different classes unlike copilot which has it all under one class. Having the reader and the solver in different classes allows them to be separate and not mess with one another. Due to the modularity of the Gemini code it also makes it easier to read and make changes compared to the copilot code. Lastly Gemini has less lines compared to Copilot

Error handling

Both handle it pretty well though Gemini could separate invalid file name and invalid file grid. They also both could handle end of file indicator better.

Comments

There are pretty much no comments on both though Gemini does include it sometimes

f. Select one to use as the basis for this assignment and justify why you picked it.

I am going to select Gemini due to the modularity and it being easier to read. Speed is not that relevant to this assignment due to the small size which is shown by the execution time tests. Both codes handle the same edge cases though both could be improved on error handling. Though Copilot did not take into account end of file indicators.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

To improve correctness I am going to edit the error handling so that it is more specified and catches end of file error

For execution time I would change it to something similar to the copilot code where I would store the empty slots and recurse off that rather than iterating through the entire code

As for maintainability I will add comments and change output to simply modify the original input file.